

กระบวนการเตรียมและสำรวจข้อมูล (EDA) บทความวิจัยจาก SCOPUS สำหรับ MACHINE LEARNING

PRESENT BY

นางสาวจตุพร ทุ่มซ = 66011212009

นางสาวเจนจิรา คำปิว 66011212080

กระบวนการการทำ EDA

ดาวน์โหลดชุดข้อมูล เป็นไฟล์ CSV จาก Scopus.com จำนวน 10 สาขาวิชา สาขาละ 600 บทความ ได้แก่ สาขา Artificial_Intelligence, Computational Theory AND Mathematics, Computer Graphics AND Computer Aided Design, Computer Networks AND Communications, Computer Vision AND Pattern Recognition, Hardware AND Architecture, Human Computer Interaction, Information Systems, Signal Processing, Software โดยใช้ keyword ที่กำหนดจากในเลือก subject area เฉพาะ Computer Science เท่านั้น

Search within Article title, Abstract, Keywords	Search documents * Artificial Intelligence	X	🗑️
AND			
Search within Article title, Abstract, Keywords	Search documents Computational Theory AND Mathematics	X	🗑️
AND			
Search within Article title, Abstract, Keywords	Search documents Computer Graphics AND Computer-Aided Design	X	🗑️
AND			
Search within Article title, Abstract, Keywords	Search documents Computer Networks AND Communications	X	🗑️
AND			
Search within Article title, Abstract, Keywords	Search documents Computer Vision AND Pattern Recognition	X	🗑️

การกำจัดข้อมูลซ้ำซ้อน (DATA DEDUPLICATION)

ในการตรวจสอบชื่อบทความ (Title) หากพบว่าบทความใดถูกจัดอยู่ในหลายสาขาวิชา (เช่น อยู่ใน AI และ Software) ระบบจะลบข้อมูลนั้นทิ้งทั้งหมด เพื่อป้องกันปัญหา ความคลุมเครือของคลาส (Class Ambiguity) ซึ่งจะทำให้โมเดล Machine Learning สับสนและลดความแม่นยำในการทำนาย

```
1 # เก็บเฉพาะบทความที่ไม่ปรากฏซ้ำใน Subject อื่น (เช็คจากชื่อเรื่อง)
2 df = df.drop_duplicates(subset=['Title'], keep=False)
3
4 print(f"จำนวนข้อมูลหลังลบรายการซ้ำ: {df.shape[0]} แถว")
```

จำนวนข้อมูลหลังลบรายการซ้ำ: 5036 แถว

การจัดการค่าว่าง (MISSING VALUE HANDLING)

1

ลบคอลัมน์ที่ไม่จำเป็น: ตรวจสอบรายละเอียดของค่าว่าง หากคอลัมน์ใดมีค่าว่างสูงเกินเกณฑ์ หรือไม่มีผลต่อการวิเคราะห์ขอความ (เช่น Volume, Issue, Page) จะถูกตัดทิ้งเพื่อลดขนาดหน่วยความจำ

2

เติมเต็มค่าว่าง (Imputation): คอลัมน์ที่จำเป็นต่อการวิเคราะห์ เช่น Author Keywords หรือ Abstract หากมีค่าว่าง จะถูกเติมด้วยค่าว่างแบบ String (Empty String " ") เพื่อไม่ให้เกิด Error ในขั้นตอนการนำขอความมาต่อกัน

การจัดการค่าว่าง (MISSING VALUE HANDLING)

```
1 import re
2 import nltk
3 from nltk.corpus import stopwords
4 nltk.download('stopwords')
5
6 # =====
7 # 1. โค้ดส่วนต้น (ใช้ของเดิมของคุณได้เลย)
8 # =====
9 columns_to_drop = [
10     'Volume', 'Issue', 'Art. No.', 'Page start',
11     'Page end', 'Open Access', 'EID', 'Link', 'Source'
12 ]
13 df_clean = df.drop(columns=columns_to_drop)
14
15 text_fill_cols = ['Author Keywords', 'Index Keywords', 'Authors', 'Author full names', 'Author(s) ID', 'DOI']
16 for col in text_fill_cols:
17     df_clean[col] = df_clean[col].fillna('')
18
19 # =====
20 # 2. โค้ดส่วนคลีนข้อความ (นำของใหม่มาวางทับตรงนี้)
21 # =====
22 stop_words = set(stopwords.words('english'))
23 custom_noise = [
24     'copyright', 'license', 'springer', 'nature', 'switzerland',
25     'authors', 'exclusive', 'pte', 'ltd', 'rights', 'reserved',
26     'ag', 'publishing', 'singapore', 'llc', 'international'
27 ]
28 stop_words.update(custom_noise)
29
30 def clean_text(text):
31     if not isinstance(text, str):
32         return ""
33     text = text.lower()
34     text = re.sub(r'copyright.*', '', text) # ตัดประโยคลิขสิทธิ์ทิ้ง
35     text = re.sub(r'^a-z\s', '', text)
36     words = text.split()
37     words = [w for w in words if w not in stop_words]
38     return " ".join(words)
39
40 print("กำลังทำความสะอาดข้อมูล Title และ Abstract...")
41 df_clean['Clean_Title'] = df_clean['Title'].apply(clean_text)
42 df_clean['Clean_Abstract'] = df_clean['Abstract'].apply(clean_text)
43 df_clean['Combined_Text'] = df_clean['Clean_Title'] + " " + df_clean['Clean_Abstract']
44
45 print("กระบวนการเสร็จสิ้น ชุดข้อมูลพร้อมสำหรับการทำ Vectorization")
46 display(df_clean[['Subject', 'Combined_Text']].head())
```

การทำความสะอาดข้อความ (TEXT PREPROCESSING)

- Case Folding: เปลี่ยนอักษรตัวพิมพ์ใหญ่ให้เป็นตัวพิมพ์เล็กทั้งหมด
- Noise Removal: ใช้ Regular Expression (Regex) ลบตัวเลข เครื่องหมายวรรคตอน และอักขระพิเศษออก
- Copyright Stripping: กำหนด Regex เพื่อสกัดและตัดประโยคลิขสิทธิ์ของสำนักพิมพ์ (เช่น ข้อความที่เริ่มด้วยคำว่า Copyright, Springer, Nature) ออกจากส่วนหางของบทความ เนื่องจากเป็นคำขยะ (Noise) ที่ไม่มีความหมายทางวิทยาศาสตร์
- Stop Word Removal: ใช้ไลบรารี NLTK กรองคำเชื่อมภาษาอังกฤษ (เช่น a, an, the, is, in) ออก เพื่อให้โมเดลโฟกัสเฉพาะคำศัพท์สำคัญ

```
26 def clean_text(text):
27     if not isinstance(text, str):
28         return ""
29     text = text.lower()
30     text = re.sub(r'copyright.*', '', text) #ตัดประโยคลิขสิทธิ์ทิ้ง
31     text = re.sub(r'^a-z\s', '', text)
32     words = text.split()
33     words = [w for w in words if w not in stop_words]
34     return " ".join(words)
35
36 print("กำลังทำความสะอาดข้อมูล Title และ Abstract...")
37 df_clean['Clean_Title'] = df_clean['Title'].apply(clean_text)
38 df_clean['Clean_Abstract'] = df_clean['Abstract'].apply(clean_text)
39 df_clean['Combined_Text'] = df_clean['Clean_Title'] + " " + df_clean['Clean_Abstract']
40
```

การทำ VISUALIZATION และความสัมพันธ์กับข้อมูล



WORD CLOUD

เป็นกราฟิกแสดงผลความถี่ของคำศัพท์ในแต่ละสาขาวิชา โดยขนาดของตัวอักษรจะสะท้อนถึงจำนวนครั้งที่คำนั้นปรากฏ การสร้าง Word Cloud แยกตามสาขาช่วยให้เห็น คุณลักษณะเฉพาะ (Feature Importance) ของแต่ละคลาสได้อย่างรวดเร็ว ว่าแต่ละวิชามีคีย์เวิร์ดใดเป็นแกนหลัก



KEYWORD CO-OCCURRENCE NETWORK (HTML)

กราฟเครือข่าย Interactive แสดงความสัมพันธ์ของการปรากฏคู่กันของคำสำคัญ

- จุด (Nodes)
- เส้นเชื่อม (Edges)
- การจัดกลุ่ม (Community Detection)

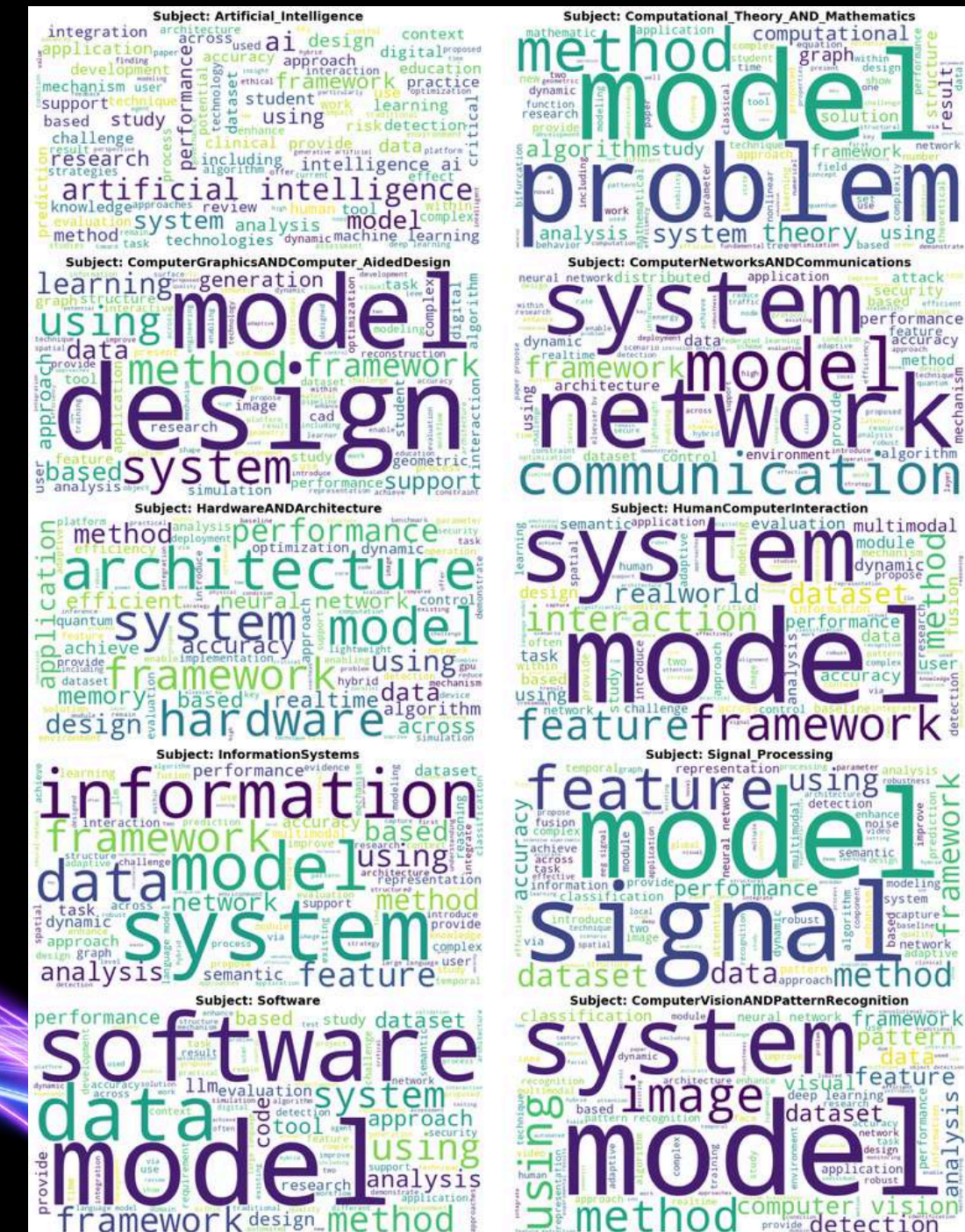


INTERACTIVE BAR CHART

กราฟแสดงสัดส่วนปริมาณบทความในแต่ละคลาสหลังจากการคลีนข้อมูล เพื่อประเมินความสมดุลของชุดข้อมูลก่อนนำไปเทรนโมเดล

การทำ VISUALIZATION และความสัมพันธ์กับข้อมูล

```
1 import matplotlib.pyplot as plt
2 from wordcloud import WordCloud
3
4 #หาชื่อสาขาวิชาทั้งหมดที่มีในชุดข้อมูล
5 subjects = df_clean['Subject'].unique()
6
7 #ตั้งค่าขนาดกราฟให้แสดง 5 แถว 2 คอลัมน์ (สำหรับ 10 สาขาวิชา)
8 fig, axes = plt.subplots(5, 2, figsize=(20, 25))
9 axes = axes.flatten()
10
11
12 for i, sub in enumerate(subjects):          #ดึงข้อความจากคอลัมน์ Combined_Text เฉพาะสาขานั้น
13     text_data = " ".join(df_clean[df_clean['Subject'] == sub]['Combined_Text'].tolist())
14
15     #สร้าง Word Cloud
16     wordcloud = WordCloud(
17         width=800,
18         height=400,
19         background_color='white',
20         max_words=100,
21         colormap='viridis',
22         random_state=42
23     ).generate(text_data)
24
25     #วาดกราฟลงในตำแหน่ง (Subplot) ที่กำหนด
26     axes[i].imshow(wordcloud, interpolation='bilinear')
27     axes[i].set_title(f'Subject: {sub}', fontsize=18, fontweight='bold')
28     axes[i].axis('off')
29
30 plt.tight_layout()    #ปรับระยะห่างระหว่างกราฟให้สวยงามและแสดงผล
31 plt.show()
```



ข้อมูลก็นำไปทำ FEATURE VECTOR

การควบรวมข้อความ (TEXT CONCATENATION)

ข้อมูลที่คลีนแล้วจาก 'Title' และ 'Abstract' ถูกนำมารวมเป็นฟีเจอร์ใหม่ชื่อ 'Combined_Text' จากนั้นจึงนำคำจาก 'Author Keywords' และ 'Index Keywords' มาต่อท้ายเพื่อสร้างเป็นคอลัมน์ 'Final_Text'

การสร้าง FEATURE VECTOR ด้วย TF-IDF

ข้อมูลใน 'Final_Text' ถูกนำมาแปลงเป็นเมตริกซ์ตัวเลขด้วยเทคนิค TfidfVectorizer ซึ่งจะประเมินน้ำหนักความสำคัญของคำศัพท์จากการปรากฏในบทความเมื่อเทียบกับความถี่ในชุดข้อมูลทั้งหมด

การเข้ารหัสคำตอบ (Label Encoding): ข้อมูลคอลัมน์ 'Subject' หรือ 10 สาขาวิชาเป้าหมาย ถูกแปลงให้อยู่ในรูปแบบหมวดหมู่ตัวเลขด้วย LabelEncoder ขั้นตอนนี้จำเป็นมากเพราะอัลกอริทึม Machine Learning ต้องอาศัยข้อมูลในรูปแบบตัวเลขเพื่อเรียนรู้รูปแบบ

การเข้ารหัสเป้าหมาย (LABEL ENCODING)

แปลงชื่อสาขาวิชาทั้ง 10 หมวดให้เป็นคลาสตัวเลข 0-9 ทำให้ชุดข้อมูลอยู่ในรูปแบบ Array ทางคณิตศาสตร์ที่พร้อมสำหรับกระบวนการ Machine Learning อย่างสมบูรณ์

ข้อมูลที่น่าไปทำ FEATURE VECTOR

```
1 from sklearn.feature_extraction.text import TfidfVectorizer
2 from sklearn.preprocessing import LabelEncoder
3
4 #นำ Keywords มาต่อท้าย Combined_Text เพื่อเพิ่มบริบทที่เข้มข้นขึ้นให้กับโมเดล
5 df_clean['Final_Text'] = df_clean['Combined_Text'] + " " + df_clean['Author Keywords'] + " " + df_clean['Index Keywords']
6
7 #กำหนดคอสัมสำหรับ Features (X) และ Target (y)
8 X_raw = df_clean['Final_Text']
9 y_raw = df_clean['Subject']
10 🧠
11 #แปลงตัวแปรตาม(y) ให้เป็นหมวดหมู่ตัวเลข 0-9
12 print("กำลังแปลง Target Variable...")
13 encoder = LabelEncoder()
14 y = encoder.fit_transform(y_raw)
15
16 #แปลงข้อความ(X) ให้เป็น Feature Vector (เมทริกซ์ตัวเลข)
17 print("กำลังสร้าง Feature Vector ด้วย TF-IDF...")
18 tfidf_vectorizer = TfidfVectorizer(
19     max_features=5000,      # เลือกคำศัพท์ที่มีความสำคัญและพบบ่อยที่สุด 5,000 คำ (ลดขนาดมิติข้อมูลป้องกันเครื่องค้าง)
20     ngram_range=(1, 2)     # ให้โมเดลเรียนรู้ทั้งคำเดี่ยว (เช่น "machine") และคำคู่ที่มักมาด้วยกัน (เช่น "machine learning")
21 )
22
23 X = tfidf_vectorizer.fit_transform(X_raw)
24
25 # ตรวจสอบผลลัพธ์
26 print("-" * 50)
27 print(f"ขนาดของ Feature Matrix (X): {X.shape[0]} ตัวอย่าง, {X.shape[1]} ฟีเจอร์ (คำศัพท์)")
28 print(f"ขนาดของ Target (y): {y.shape[0]} ตัวอย่าง")
29 print("รายชื่อคลาสที่ถูกแปลงเป็นตัวเลข:", encoder.classes_)
```



THANK YOU

FOR YOUR TIME AND ATTENTION

